2024 IEEE International Conference on Systems, Man, and Cybernetics (SMC)
October 6-10, 2024. Sarawak, Malaysia

Autonomous Structural Inspection with an Octocopter UAV: Integration of 3D LiDAR and Gimbal-Mounted Camera

Ji-Hwan Son[1,*], Deokyun Kim[1], and Jihun Cha[1]

Abstract— **This paper presents an autonomous inspection system that utilizes an octocopter UAV to enhance the structural safety assessments of challenging environments such as bridges, high-rise buildings, and dams. The system features an octocopter equipped with a rotating 3D LiDAR for comprehensive environmental data collection, a three-axis gimbal-mounted camera for high-resolution imaging, and an onboard computer for data processing and control. Our algorithms are designed to recognize environmental structures, identify planar surfaces, and calculate optimal gimbal poses for detailed inspections. The UAV autonomously navigates and performs these tasks, ensuring precise structural assessments. We validate the effectiveness of this system through experiments with the octocopter UAV.**

I. Introduction

In recent years, the use of unmanned aerial vehicles (UAVs) has increased significantly in various fields owing to the development of related technologies, including hardware and algorithms. UAVs can navigate dense forests [1], explore unknown tunnel [2], and reconstruct certain areas [3] using various sensors, such as camera and LiDAR. Moreover, UAVs are gradually becoming more closely connected to our lives. It can perform aerial delivery [4], precision agriculture [5], search and rescue [6], and other related tasks.

Unlike ground robots that can only move on a plane, UAVs have six degrees of freedom (DOFs) and can fly near the ground and higher in the sky. Multirotor UAVs can only be controlled in four directions, such as roll, pitch, yaw, and throttle, instead of the six available DOFs. Although there are restrictions for generating movements in the six DOFs, flexible movements have been demonstrated by the development of UAV control algorithms. Thus, the use of UAVs has drastically increased in recent years. In particular, UAVs have shown their strengths in inspection for structural safety inspection, which is generally performed by humans. UAVs can easily fly into spaces that are not easily accessible to humans. Moreover, based on their ability to perform precise examinations using various sensors, UAVs can be more capable than humans inspectors.

As construction of complex structures such as bridges, tall buildings, and dams continues to increase, so does the demand for inspections in challenging and hazardous environments to meet maintenance standards and regulatory requirements. These structures are often difficult and dangerous for human access and require significant maintenance. By equipping unmanned aerial vehicles (UAVs) with cameras and various sensors, inspections can be conducted autonomously, allowing for diverse and efficient task adaptation

UAV-based inspections are typically performed either through manual control by an operator guiding the UAV towards specific targets or via pre-planned inspection routes. For truly autonomous operations, critical functionalities such as obstacle avoidance, target recognition, and inspection route planning must be integrated to complete the mission effectively. Research on autonomous inspection has extensively covered various targets, including solar panels [7], dams [8], power lines [9], bridges [10], historical buildings [11], tall telecommunication towers [12], railroads [13], utility lines [14], and aircraft [15]. Furthermore, integration of diverse sensors into the inspection process [16] have been explored. In our work, we continue this trend by integrating sophisticated hardware systems and algorithms to enhance the capabilities of autonomous inspections.

To capture comprehensive spatial information of the inspection environment, our approach involves equipping an octocopter UAV with a module that continuously rotates a 3D LiDAR. This setup enables the UAV to gather complete environmental data. The collected LiDAR data are then processed into point clouds, which are subsequently transformed into plane information within a designated region of interest (ROI) to accurately identify the inspection targets. Based on these identified planes, poses are generated to facilitate the acquisition of high-resolution images. Route planning for the UAV is then executed using a cost function derived from these poses. Following this planning phase, the octocopter autonomously recognizes the targets within the ROI, establishes the necessary poses for detailed inspection, and captures high-resolution images by navigating according to the pre-planned route.

The structure of this paper is as follows: In Section II, we elaborate on the hardware used in our autonomous inspection system, including the dynamics of the octocopter UAV and the data processing techniques applied to the outputs of the rotating 3D LiDAR, which collectively facilitate the acquisition of point clouds of the surrounding area. Section III details our methodology for estimating and managing planes and generating inspection poses. To validate our proposed system, we conducted real-world experiments, the setup and outcomes of which are discussed in Section IV. Finally, conclusions are drawn in Section V.

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. NRF-2020M3C1C2A01080836)

[1]Autonomous UAV Research Section, Digital Convergence Research Laboratory, Electronics and Telecommunications Research Institute (ETRI), South Korea jihwan@etri.re.kr deokyunkim@etri.re.kr jihun@etri.re.kr

2024 IEEE International Conference on Systems, Man, and Cybernetics (SMC) | 978-1-6654-1020-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/SMC54092.2024.10831954

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on March 30,2026 at 21:36:27 UTC from IEEE Xplore. Restrictions apply.

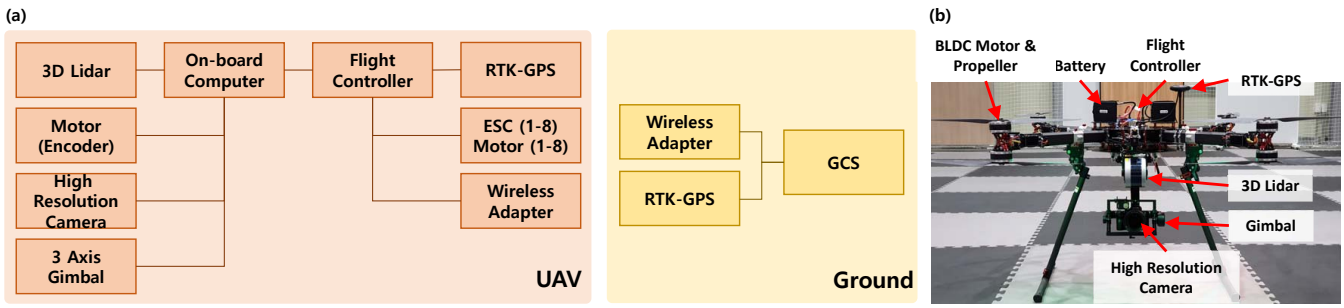

Fig. 1. Octocopter UAV for inspection. Diagram of autonomous inspection system (left) and photograph of octocopter UAV with its components(right)

II. Octocopter UAV based Autonomous Inspection System

A. Hardware of the UAV

We propose an autonomous inspection system using an octocopter UAV. The octocopter can generate sufficient thrust to carry various devices such as a rotating 3D LiDAR and motor with an encoder for continuous rotation, high-resolution camera, three-axis gimbal for camera viewpoint adjustment, and onboard computer for real-time data processing and control. Fig. 1-(a) illustrates the structure and interface of the developed autonomous inspection system.

The octocopter UAV is equipped with eight 18-inch propellers and brushless DC motors, and the distance between each motor is 1200 mm. It operates using two 6S 16000 mAh batteries connected in series. For enhanced navigational accuracy, the UAV incorporates an RTK (real-time kinematic)-GPS (global positioning system) module, offering superior precision over standard GPS systems. The position information received from the RTK-GPS has a low update rate, making it insufficient for controlling the UAV and acquiring real-time point cloud data. To address this issue, the flight controller uses the Extended Kalman Filter (EKF) to estimate the position at a higher frequency by integrating data from internal accelerometers, gyros, and magnetometer sensors. This estimated position is then used not only for UAV control but also for combining with real-time point cloud data. The complete setup of the octocopter UAV and its components is illustrated in Fig. 1-(b).

B. Octocopter UAV dynamics

We define the generalized coordinates of the octocopter UAV as $p = \{x, y, z\}^T \in \mathbb{R}^3$ with Euler angles $q = \{\phi, \theta, \psi\}^T \in \mathbb{R}^3$. Components ϕ, θ, and ψ represent the roll, pitch, and yaw, respectively, and have the following ranges: $-\frac{\pi}{2} \leq \phi \leq \frac{\pi}{2}$, $-\frac{\pi}{2} \leq \theta \leq \frac{\pi}{2}$, and $-\pi \leq \psi < \pi$. The rotation matrices for the Euler angles are defined as $R_{x,\phi}$, $R_{y,\theta}$, and $R_{z,\psi}$.

Total external force F acting on the octocopter UAV consists of translational force F_e and rotational force τ. The external force is generated by L_k from the rotating propellers, where $L_k = (\omega_k)^2$, and ω_k is the angular velocity of the k-th propeller of the coaxial rotor. We define the input matrices for control as $U = \{U_1, U_2, U_3, U_4\}^T \in \mathbb{R}$. Subsequently, total thrust τ and the torque of each Euler angle, $(\tau_\phi, \tau_\theta, \tau_\psi)$, can be expressed as where l is the distance between the center of mass of the octocopter UAV and its motor, c_τ is the thrust coefficient, and c_d is the drag coefficient. The input of the octocopter generated by the thrust force of each motor becomes

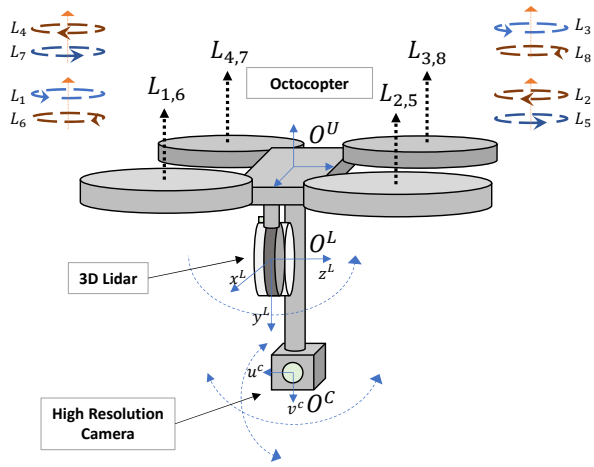

Fig. 2. Octocopter and its local frames

$$U = \begin{bmatrix} U_1 \\ U_2 \\ U_3 \\ U_4 \end{bmatrix} = \begin{bmatrix} \tau \\ \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix} = \begin{bmatrix} c_\tau(\sum_{k=1}^{8} L_k) \\ c_\tau l(-L_2 + L_4 - L_5 + L_7) \\ c_\tau l(L_1 - L_3 + L_6 - L_8) \\ c_d(\sum_{k=1}^{8}(-1)^{k-1} L_k) \end{bmatrix} \quad (1)$$

The translation force is only related to the total thrust generated by the propellers and is given by

$$\begin{bmatrix} m\ddot{x} \\ m\ddot{y} \\ m\ddot{z} \end{bmatrix} = R_{z,\psi} R_{y,\theta} R_{x,\phi} \begin{bmatrix} 0 \\ 0 \\ U_1 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix} \quad (2)$$

where m is the mass of the octocopter UAV and g is the acceleration of gravity. This function can be simplified as

$$\begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = \frac{U_1}{m} \begin{bmatrix} \mathrm{s}\phi\mathrm{s}\psi - \mathrm{c}\phi\mathrm{s}\theta\mathrm{c}\psi \\ \mathrm{s}\phi\mathrm{c}\psi + \mathrm{c}\phi\mathrm{s}\theta\mathrm{s}\psi \\ \mathrm{c}\phi\mathrm{c}\theta \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} \quad (3)$$

The rotational force of the coaxial rotor is given by

$$\begin{aligned} \tau &= J\ddot{q} + \frac{d}{dt} J\dot{q} - \frac{1}{2}\frac{\partial}{\partial t}\dot{q}^T J\dot{q} \\ &= J\ddot{q} + C(q, \dot{q})\dot{q} \end{aligned} \quad (4)$$

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on March 30,2026 at 21:36:27 UTC from IEEE Xplore. Restrictions apply.

where J is the inertia matrix and $C(q, \dot{q})$ is the Coriolis term. The equation above can be rewritten as $J\ddot{q} = \tau - C(q, \dot{q})\dot{q}$. Then, we can modify the equation as follows:

$$\begin{bmatrix} \ddot{\phi} \\ \ddot{\theta} \\ \ddot{\psi} \end{bmatrix} = \begin{bmatrix} \frac{1}{J_{xx}} & 0 & 0 \\ 0 & \frac{1}{J_{yy}} & 0 \\ 0 & 0 & \frac{1}{J_{zz}} \end{bmatrix} \begin{bmatrix} \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix} - \begin{bmatrix} \dot{\theta}\dot{\psi}\left(\frac{J_{yy}-J_{zz}}{J_{xx}}\right) \\ \dot{\phi}\dot{\psi}\left(\frac{J_{zz}-J_{xx}}{J_{yy}}\right) \\ \dot{\theta}\dot{\phi}\left(\frac{J_{xx}-J_{yy}}{J_{zz}}\right) \end{bmatrix}$$

where J_{xx}, J_{yy}, and J_{zz} represent the moments of inertia with respect to the x, y, and z axes, respectively. The Coriolis effect on the rotational force has a marginal influence. Therefore, the dynamics of the rotational part can be simplified as follows:

$$\begin{bmatrix} \ddot{\phi} \\ \ddot{\theta} \\ \ddot{\psi} \end{bmatrix} \simeq \begin{bmatrix} \frac{\tau_\phi}{J_{xx}} \\ \frac{\tau_\theta}{J_{yy}} \\ \frac{\tau_\psi}{J_{zz}} \end{bmatrix} \tag{5}$$

Using the above equations, the octocopter UAV can generate force by combining the thrust of the motors. Using proportional–integral–derivative controllers, the octocopter can fly with six DOFs and flexibly complete its inspection mission. We use an open-source flight controller (PX4 [17]) based on these principles and modify the parameters of the flight controller for stable flight of the octocopter UAV.

C. Point cloud generation using rotating 3D LiDAR

To prevent collisions, accurate geometry of target structures and their surroundings must be determined. For this purpose, we have integrated a hardware module that captures comprehensive environmental data using a continuously rotating LiDAR. While conventional 3D LiDARs can scan 360° horizontally, they are typically limited in their vertical scanning capability. Specifically, the Velodyne puck LiDAR we use has a vertical field of view of only 30°. To address this limitation, we mount the 3D LiDAR vertically and rotate it 360° to ensure full environmental coverage. The rotation is facilitated by a motor equipped with an encoder that precisely measures the rotation angle. Additionally, a slip ring is installed between the LiDAR and the motor, enabling the continuous transmission of data and power to the LiDAR

The sensing module, equipped with a 3D LiDAR, is mounted on the front of the octocopter UAV. This LiDAR continuously rotates to measure distances in all directions, except for the area obscured by the UAV's body. Given the UAV's capability to navigate with six degrees of freedom (DOFs), rapid changes in its attitude and position necessitate that LiDAR data be immediately transformed into a global coordinate system.

To facilitate this, we define a rotation matrix R_A^B that rotates from frame A to frame B, and we establish the initial coordinate system as an Earth-fixed world coordinate system. In this system, the x, y, and z axes correspond to the east, north, and up directions, respectively. The initial position and attitude of the 3D LiDAR, located at a distance d^L from the UAV's body frame, are depicted in Fig. 2. The LiDAR, which is mounted vertically and rotated by the motor, continuously receives an angle ψ^L from encoder of the motor. This setup allows the following calculations to be performed for the LiDAR data:

$$\begin{aligned} p^{L'} &= R_{z,\psi^L} R_{x,\phi^L} p^L \\ &= \begin{bmatrix} c\psi^L & -c\phi^L s\psi^L & s\phi^L s\psi^L \\ s\psi^L & c\phi^L c\psi^L & -s\phi^L c\psi^L \\ 0 & s\phi^L & c\phi^L \end{bmatrix} \begin{bmatrix} x^L \\ y^L \\ z^L \end{bmatrix} \end{aligned} \tag{6}$$

where ϕ^L and ψ^L are the Euler angles of the LiDAR and $\phi^L = -\frac{\pi}{2}$. Then, we can simplify the equation as follows:

$$p^{L'} = \begin{bmatrix} c\psi^L & 0 & -s\psi^L \\ s\psi^L & 0 & c\psi^L \\ 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} x^L \\ y^L \\ z^L \end{bmatrix} \tag{7}$$

Based on the above equations, data from the rotating 3D LiDAR can be used to obtain distance information for the entire environment. As the 3D LiDAR is attached to the octocopter, its data must be converted from the body frame of the octocopter to the world coordinate frame. To change the frame, we must transfer the LiDAR data to the coordinates of octocopter using the following rotation matrix:

$$\begin{aligned} R_{L'}^W &= R_{x,\phi} R_{y,\theta} R_{z,\psi} \\ &= \begin{bmatrix} c\theta c\psi & -c\theta s\psi & s\phi \\ s\psi s\theta c\psi + c\phi s\psi & -s\phi s\theta s\psi + c\phi c\psi & -s\phi c\theta \\ -c\phi s\theta c\psi + s\phi s\psi & c\phi s\theta s\psi + s\phi c\psi & c\phi c\theta \end{bmatrix} \end{aligned} \tag{8}$$

The LiDAR data can then be expressed in the world coordinates as follows:

$$p^{L,W} = R_{L'}^W p^{L'} + R_{L'}^W d^L \tag{9}$$

Finally, the LiDAR data are converted into a point cloud and used to obtain structural information for inspection.

III. Point-cloud-based Environment Detection and Planning for Autonomous Inspection

A. Point-cloud-based Plane Detection and Calculations

To detect planes within the point clouds generated by our rotating 3D LiDAR mounted on the octocopter UAV, we initially employ a region-growing segmentation algorithm. This algorithm identifies similarities among pointcloud to form planar segments. However, the algorithm cannot deal with pointcloud that change gradually, a common occurrence as the octocopter moves and continuously collects pointcloud. To address this issue, we have refined our methodology to include a region-growing segmentation technique, which clusters the pointcloud by comparing the normal vectors of each point cloud. To enhance adaptability of the pointcloud, we modify the algorithm to

To enhance adaptability to evolving pointcloud data, we modify the algorithm to incorporate comparison and merging processes. These improvements enable more accurate and reliable plane recognition and clustering even in continuously changing pose of the UAV, as detailed in Algorithm 1.

Let define a plane group of k-th point cloud G_k and its individual points as G_k^i for $i = \{1, 2, \cdots, l\}$, where $G_k^i =$

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on March 30,2026 at 21:36:27 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Plane recognition and clustering

```
while Operation enabled do
    Filter and downsample point cloud
    if G == ∅ then
        Run region-growing segmentation
        Compute Algorithm 2
    else
        Set unsorted point-cloud group as G_unsorted
        for k = 1:the number of plane groups do
            for j = 1:the number of G_unsorted do
                if G^j_unsorted is in boundary of G_k then
                    G_k ← G^j_unsorted
                    Delete G^j_unsorted
                else
                    Calculate Eq. (11) and (12)
                    if |ρ_k| ≤ ρ and |η_kj| ≤ η then
                        G_k ← P^j
                        Delete P^j
                    end if
                end if
            end for
        end for
        if G_unsorted ≠ ∅ then
            Run region-growing segmentation
            if Plane group is generated then
                Compute Algorithm 2
            end if
            Remove G_unsorted
        end if
    end if
end while
```

$\{x, y, z\}^T \in \mathbb{R}^3$. Its center of the point cloud G_k^C can be calculated as

$$G_k^C = \frac{1}{l}\sum_{i=1}^{l} G_k^i \tag{10}$$

where $G_k^C = \{x, y, z\}^T \in \mathbb{R}^3$.

Based on the group of planes, we can obtain the normal vector of the k-th plane, n_k, and its distance from the center, ρ_k, where $n_k = \{n_k^x, n_k^y, n_k^z\}^T \in \mathbb{R}^3$ and $\rho_k \in \mathbb{R}$. Then, each plane can be generated as the following function when $\rho_k = 0$:

$$n_k^x x + n_k^y y + n_k^z z + d_k = \rho_k \tag{11}$$

Using the normal vector of each plane, we can calculate the similarity between two planes as follows:

$$\eta_{jk} = n_j n_k^T \tag{12}$$

$\eta_{jk} \in \mathbb{R}$. Using the Eq. 11 and 12, Algorithm 3 determines whether the given planes are considered identical based on predefined parameters ρ and η.

To obtain the boundaries of each plane in the same coordinate system, each plane is rotated by its normal vector.

Algorithm 2 Comparison between obtained planes

```
for k = 1:the number of planes do
    Calculate basic information of plane k
end for
for k = 1:the number of planes do
    for j = 1:the number of planes do
        if k ≠ j and j ≠ ∅ then
            Calculate Eq. (11) and (12)
            if |ρ_k| ≤ ρ and |η_kj| ≤ η then
                Add plane j into k and remove plane k
                Update current center of plane
            end if
        end if
    end for
end for
```

$$\begin{aligned} G_k^{i'} &= R_{z,\psi_k} R_{y,\theta_k}(G_k^i - G_k^C) \\ &= \begin{bmatrix} c\theta_{G_k} c\psi_{G_k} & -c\theta_{G_k} s\psi_{G_k} & s\theta_{G_k} \\ s\psi_{G_k} & c\psi_{G_k} & 0 \\ -s\theta_{G_k} c\psi_{G_k} & s\theta_{G_k} s\psi_{G_k} & c\theta_{G_k} \end{bmatrix} (G_k^i - G_k^C) \end{aligned} \tag{13}$$

where the Euler angle of G_k for rotation can be obtained as follows:

$$\theta_{G_k} = \text{atan2}(n_k^y, n_k^x) \tag{14}$$

$$\psi_{G_k} = \text{atan2}(n_k^z, \sqrt{(n_k^x)^2 + (n_k^y)^2}) \tag{15}$$

After rotating the k-th point-cloud group, the boundary of the plane can be obtained by comparing the point-cloud values. The obtained boundary is represented as $[G_k^{x,min}, G_k^{x,max}]$, $[G_k^{y,min}, G_k^{y,max}]$, and $[G_k^{z,min}, G_k^{z,max}]$.

B. Pose generation for inspection

When using a camera for inspection, location or navigation errors can lead to missing the inspection target. To mitigate this risk, we established an overlapping area by setting an overlapping distance, ζ. This approach ensures that each captured image covers an area of 1.8 meters in width and 1 meter in length. Consequently, the number of required inspection poses is determined based on the horizontal and vertical distances covered by the plane data extracted from the inspection target.

Let define a set of poses P_k of k-th point cloud G_k and its individual poses as P_k^i for $i = \{1, 2, \cdots, l\}$, where $P_k^i = \{x, y, z, \phi, \theta, \psi\}^T \in \mathbb{R}^6$. Using the algorithms, we extracted information about the square-shaped planes. Consequently, the number of poses per inspection direction can be generated by utilizing the boundaries of the plane as follows:

$$b_k^w = \lceil \frac{G_k^{y,max} - G_k^{y,min}}{c_u - \zeta} \rceil \tag{16}$$

$$b_k^h = \lceil \frac{G_k^{z,max} - G_k^{z,min}}{c_v - \zeta} \rceil \tag{17}$$

where $\lceil\rceil$ denotes the ceiling function c_u and c_v are wide and high of the visible area of the inspection target.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on March 30,2026 at 21:36:27 UTC from IEEE Xplore. Restrictions apply.

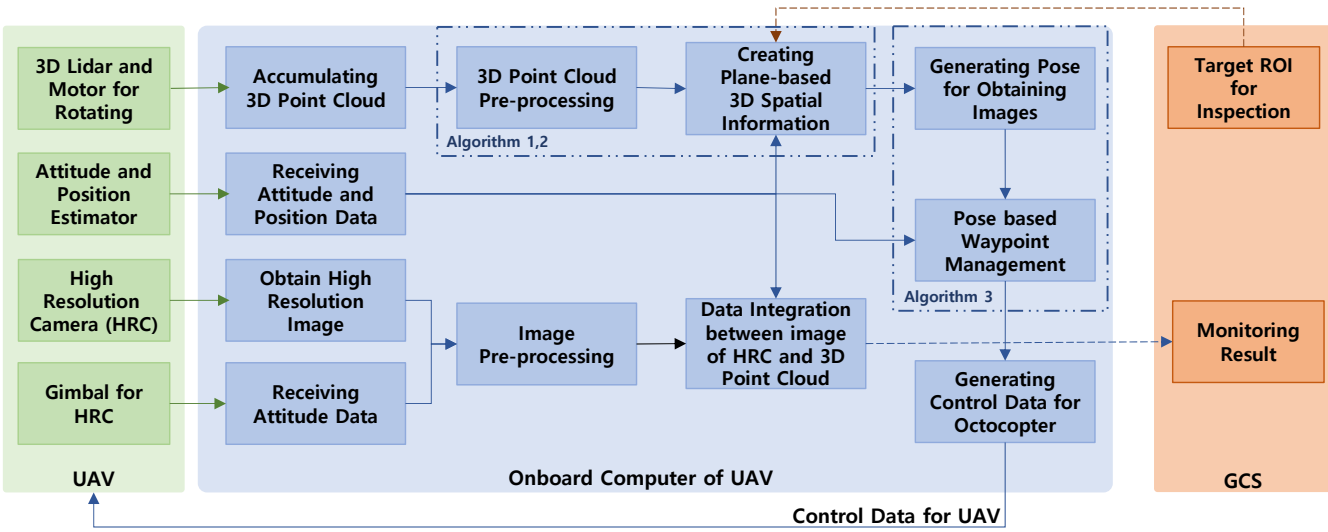

Fig. 3. Operation flowchart of inspection system

Algorithm 3 Plane-based pose generation

```
for k = 1: the number of planes do
    if P_k == ∅ or Boundary of G_k is updated then
        if Pose data is empty then
            Calculate Eq. (16) and (17)
            Generate poses based on boundary
        else
            Add poses for newly obtained area
        end if
        for P_k^j where j = 1: the number of poses do
            if Target plane is not empty then
                if Obstacle exist near P_k^j then
                    Find alternative pose
                end if
            else
                Erase P_k^j
            end if
        end for
        for P_k^j where j = 1:the number of poses do
            if P_k^j needs update then
                Find nearest available pose
                Update position of selected pose
            end if
        end for
        Sort poses based on cost function
    end if
end for
```

Using the functions above, we are able to generate poses to effectively cover a plane. However, these poses may sometimes conflict with obstacles or come too close to the ground. To address these challenges, we determine alternative poses, as detailed in Algorithm 3. Using the obtained b_k^w and b_k^h, $b_k^w \times b_k^h$ poses can be generated within the area. These poses are then processed by Algorithm 3, where they may be deleted or reassigned as necessary. The previously calculated poses were rotated and translated through Eq. 13. To map the obtained poses onto the actual plane, they are rotated and translated back to the original plane

When capturing images with a camera, increasing the aperture not only focuses the object but also enhances the clarity of the surrounding areas, making them appear as if they were in focus. This allows for a certain flexibility in inspection tasks; even if the actual distance of the plane differs from the estimated one, adjusting the aperture can ensure the entire area is effectively treated as a plane. Moreover, the algorithm is capable of estimating not only flat but also curved and circular planes by varying the aperture settings.

Based on the current relative position and target area, the algorithm generates a plan to visit the poses sequentially. Pose sorting is based on a distance cost function to minimize the plan cost. After the plan for visiting the poses is complete, the octocopter UAV flies toward the target poses sequentially to capture an inspection picture per pose.

IV. Experiment

A. Experimental setup

To evaluate our autonomous inspection system experimentally, we equipped it with a Sony A7R3 camera, featuring a 24-105 mm f/4 lens and a high resolution of 7952 × 5304 pixels. When capturing images at the 105 mm focal length, the camera achieves a field of view of 19.46° horizontally and 13.04° vertically. For instance, when an image is taken from 7 meters away from the target, the visible area of the inspection target measures 2.4 meters wide (c_u) and 1.6 meters high (c_v). These dimensions translate to a resolution that allows the detection of cracks as small as 0.3 mm. To enhance the precision of the inspections, the distance to the target may be decreased, or a camera with higher resolution may be employed. During the experiments, we maintained

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on March 30,2026 at 21:36:27 UTC from IEEE Xplore. Restrictions apply.

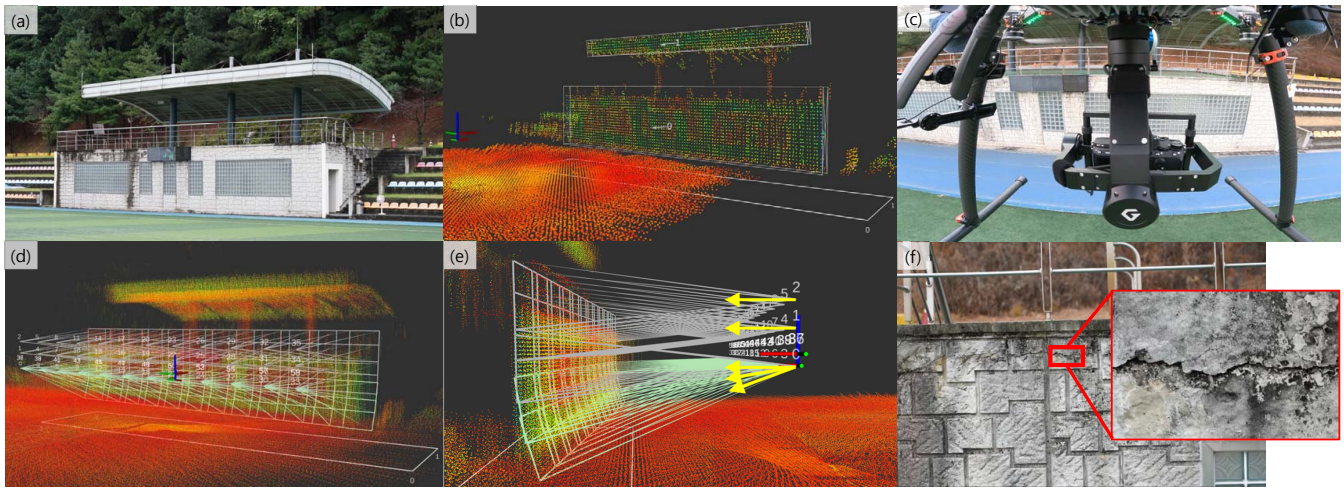

Fig. 4. (a) Target for inspection. (b) Obtained point-cloud and estimated planes. (c) Photograph of bottom camera during inspection. (d) Acquired point clouds and generated inspection poses. (e) Side view of generated inspection poses. (f) Photograph from experiment and magnified view of a crack.

an overlapping distance ζ of 0.6 meters between sequential images to ensure comprehensive coverage.

The onboard computer managed the hardware system via algorithms implemented in the Robot Operating System (ROS) running on a Linux Ubuntu platform. To verify the effectiveness of our system and these algorithms, we conducted an experiment on a sports field. The experimental target, positioned in front of a podium, was a decades-old structure with visible cracks, offering a realistic scenario for inspection.

We defined the region of interest (ROI) at the target location using Qgroundcontrol [18]. Following the initialization of the octocopter UAV, we allowed time for the RTK-GPS module to achieve stable accuracy. Once stability was confirmed, the UAV was manually navigated to a predetermined altitude in position mode. At this point, offboard control mode was activated, allowing the UAV to fly autonomously and execute the inspection tasks as directed by our developed algorithms.

B. Experimental results

Upon activating the offboard control mode, the octocopter UAV commenced data acquisition using the rotating 3D LiDAR, capturing the surrounding environment's point-cloud data. The plane areas within the region of interest (ROI) were then extracted, as illustrated in Fig. 4-(b). Based on the obtained planes, 60 inspection poses were generated as shown in Fig. 4-(d) and 4-(e). For poses close to the ground, the position and attitude were revised to comply with the low-altitude restriction.

It was confirmed that the lower inspection poses were appropriately adjusted by the algorithm to comply with low-altitude restriction. Without the constraint, two additional lines of poses would have been generated. However, to adapt to the altitude limitations, the angles of these poses were modified by our proposed algorithm to optimize image capture. As a result, two distinct angles were integrated into the lowest inspection line as illustrated in Fig. 4-(e).

Using the generated inspection poses, a simple planning algorithm calculated a route for visiting these poses based on a distance cost function. Along this route, the octocopter UAV successfully captured 60 high-resolution images, as shown in Fig. 5-(a). After the mission was completed, offboard control mode was deactivated, and the UAV was manually landed in position mode. The trajectory of the experiment is illustrated in Fig. 6 and 7.

Fig. 5. Photographs of inspection target. (a) Sixty merged photographs obtained from inspection experiment and (b) photograph of entire target

V. Conclusion

We present an autonomous inspection system that utilizes a rotating 3D LiDAR mounted on an octocopter UAV to gather environmental data in the form of point clouds. These point clouds are then processed to extract plane data, from which inspection poses are generated. The octocopter UAV autonomously navigates to these poses in sequence, capturing high-resolution images for inspection. Notably, our current research does not cover the identification of cracks within the inspection results. Various methods for detecting cracks can be found in [19], and we are developing our own algorithm tailored for an embedded board. In future work, we aim to integrate these methods to enhance the practicality of our inspection results.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on March 30,2026 at 21:36:27 UTC from IEEE Xplore. Restrictions apply.

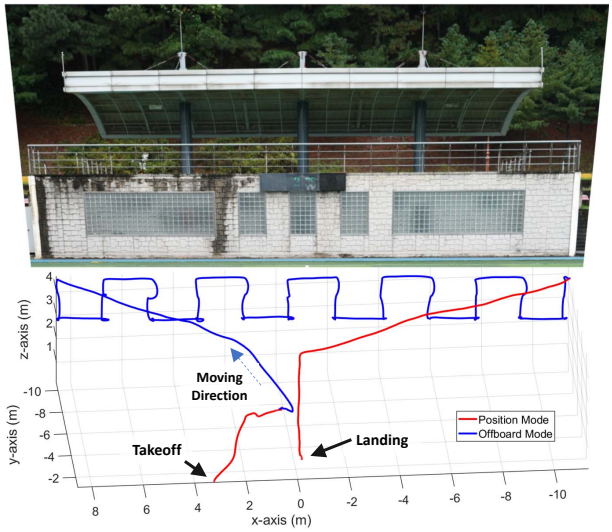

Fig. 6. Trajectory of experiment: 3D trajectories with target (red line, position mode; blue line, offboard mode)

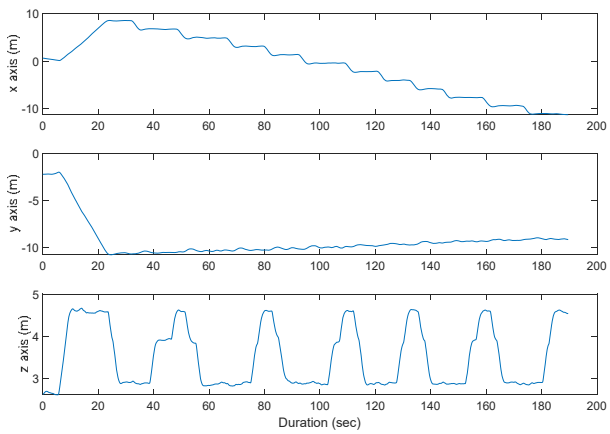

Fig. 7. Octocopter UAV trajectory along each axis during inspection experiment (offboard mode)

References

[1] X. Liu, G. V. Nardari, F. C. Ojeda, Y. Tao, A. Zhou, T. Donnelly, C. Qu, S. W. Chen, R. A. Romero, C. J. Taylor *et al.*, "Large-scale autonomous flight with real-time semantic slam under dense forest canopy," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 5512–5519, 2022.

[2] M. Tranzatto, F. Mascarich, L. Bernreiter, C. Godinho, M. Camurri, S. Khattak, T. Dang, V. Reijgwart, J. Loeje, D. Wisth *et al.*, "Cerberus: Autonomous legged and aerial robotic exploration in the tunnel and urban circuits of the darpa subterranean challenge," *arXiv preprint arXiv:2201.07067*, 2022.

[3] M. Maboudi, M. Homaei, S. Song, S. Malihi, M. Saadatseresht, and M. Gerke, "A review on viewpoints and path planning for uav-based 3d reconstruction," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 2023.

[4] M. Sajid, H. Mittal, S. Pare, and M. Prasad, "Routing and scheduling optimization for uav assisted delivery system: A hybrid approach," *Applied Soft Computing*, vol. 126, p. 109225, 2022.

[5] P. Radoglou-Grammatikis, P. Sarigiannidis, T. Lagkas, and I. Moscholios, "A compilation of uav applications for precision agriculture," *Computer Networks*, vol. 172, p. 107148, 2020.

[6] E. T. Alotaibi, S. S. Alqefari, and A. Koubaa, "Lsar: Multi-uav collaboration for search and rescue missions," *IEEE Access*, vol. 7, pp. 55 817–55 832, 2019.

[7] S. Kshetrimayum, J. J. Liou, and Y.-P. Huang, "A deep learning based detection of bird droppings and cleaning method for photovoltaic solar panels," in *2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, 2023, pp. 3398–3403.

[8] S. Zhao, F. Kang, J. Li, and C. Ma, "Structural health monitoring and inspection of dams based on uav photogrammetry with image 3d reconstruction," *Automation in Construction*, vol. 130, p. 103832, 2021.

[9] Z. Li, Q. Wang, T. Zhang, C. Ju, S. Suzuki, and A. Namiki, "Uav high-voltage power transmission line autonomous correction inspection system based on object detection," *IEEE Sensors Journal*, 2023.

[10] H. Ma, Z. Wang, Z. Shen, H. Zhang, C. Li, and F.-Y. Wang, "A multimodal approach for bridge inspection," in *2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, 2023, pp. 4649–4654.

[11] D. Smrcka, T. Baca, T. Nascimento, and M. Saska, "Admittance force-based uav-wall stabilization and press exertion for documentation and inspection of historical buildings," in *2021 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2021, pp. 552–559.

[12] D. Ribeiro, R. Santos, A. Shibasaki, P. Montenegro, H. Carvalho, and R. Calçada, "Remote inspection of rc structures using unmanned aerial vehicles and heuristic image processing," *Engineering Failure Analysis*, vol. 117, p. 104813, 2020.

[13] Y. Wu, P. Chen, Y. Qin, Y. Qian, F. Xu, and L. Jia, "Automatic railroad track components inspection using hybrid deep learning framework," *IEEE Transactions on Instrumentation and Measurement*, 2023.

[14] L. G. Librado, C. Y. Cahig, C. A. G. Banglos, J. C. Pao, and C. J. O. Salaan, "Meshing and insulation strategies for shelled uavs in proximal inspection of distribution utility lines," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 5278–5285, 2022.

[15] M. Bugaj, A. Novák, A. Stelmach, and T. Lusiak, "Unmanned aerial vehicles and their use for aircraft inspection," in *2020 New Trends in Civil Aviation (NTCA)*. IEEE, 2020, pp. 45–50.

[16] S. Asadzadeh, W. J. de Oliveira, and C. R. de Souza Filho, "Uav-based remote sensing for the petroleum industry and environmental monitoring: State-of-the-art and perspectives," *Journal of Petroleum Science and Engineering*, vol. 208, p. 109633, 2022.

[17] "Px4 autopilot: Open source autopilot for drones." [Online]. Available: https://px4.io/

[18] Qgroundcontrol. [Online]. Available: http://qgroundcontrol.com/

[19] R. Ali, J. H. Chuah, M. S. A. Talip, N. Mokhtar, and M. A. Shoaib, "Structural crack detection using deep convolutional neural networks," *Automation in Construction*, vol. 133, p. 103989, 2022.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on March 30,2026 at 21:36:27 UTC from IEEE Xplore. Restrictions apply.